

LAB GUIDE WITH SOLUTIONS

Redis Replication Labs

Six hands-on labs in Docker, with solutions inline

Module 07 · NoSQL & Redis

ICICI Bank Learning Academy

Enterprise Data Engineering with SQL, Redis & Kafka

Module 07 · NoSQL & Redis · Instructor-led · For classroom use

How to use this lab guide

Six labs, sized for a three-hour hands-on session on Redis replication - wiring a primary and replicas together, full and partial sync, running the topology under docker-compose, and manual failover. Every lab states its objective, shows the exact commands, and shows the output you should see. Solutions are inline - this document is the instructor copy and the participant copy at the same time.

All commands are **Docker-native**, and every container talks to every other one by **name**, never by IP - that choice is the subject of Lab R.4 and the reason the whole set of labs keeps working across container restarts.

Lab	Title	Covers	Time
R.1	Wire a primary and a replica, watch a full sync	Deck 7 §1	30 min
R.2	Read-only replicas, and WAIT for durability	Deck 7 §2	25 min
R.3	Partial sync after a network blip	Deck 7 §1	20 min
R.4	docker-compose: a 3-node topology by name	Deck 7 §2	30 min
R.5	Manual failover: promote a replica	Deck 7 §3	30 min
R.6	The old primary rejoins, and why it costs a full resync	Deck 7 §3	20 min

WATCH OUT Every key created in these labs starts with `icici:`. These labs build one running topology across R.1 to R.6 - do them in order in a single session, or restart from Lab R.1 if you come back later.

READ THIS Conventions: lines starting with `$` run in your own shell. Lines with no prefix run inside `redis-cli`. Indented lines in teal are the output you should see - they were captured from a real run, not written by hand.

LAB R.1 · 30 MINUTES

Wire a Primary and a Replica, Watch a Full Sync

Objective	Prerequisite
Turn two plain containers into a primary and a replica with one command, and confirm the data actually copies over.	Docker Desktop running. No prior lab needed.

Step 1 · A network the containers can find each other on

Container names only resolve to each other on a **user-defined** network - the legacy default bridge does not do this.

```
$ docker network create icici-repl-net
```

Step 2 · Start the primary

```
$ docker run -d --name redis-primary \
  --network icici-repl-net -p 7001:6379 \
  redis:7-alpine redis-server --save 60 1000
$ docker exec redis-primary redis-cli PING
PONG
```

Step 3 · Start a replica of it

```
$ docker run -d --name redis-replica1 \
  --network icici-repl-net -p 7002:6379 \
  redis:7-alpine redis-server --replicaof redis-primary 6379
```

That single flag is the entire replication setup on the replica's side. Nothing needs to be configured on the primary - it starts accepting replicas the moment one asks.

Step 4 · Confirm the link, on both sides

```
$ docker exec redis-primary redis-cli INFO replication
role:master
connected_slaves:1
slave0:ip=172.18.0.3,port=6379,state=online,offset=14,lag=1
master_repl_offset:14

$ docker exec redis-replica1 redis-cli INFO replication
role:slave
master_host:redis-primary
master_link_status:up
slave_repl_offset:14
```

Step 5 · Write on the primary, read on the replica

```
$ docker exec redis-primary redis-cli MSET \
  icici:bal:9003 341244.80 icici:bal:7711 89210.50
OK
$ docker exec redis-primary redis-cli SET icici:repl:test "written on primary"
OK

$ docker exec redis-replica1 redis-cli GET icici:repl:test
```

```
"written on primary"
$ docker exec redis-replica1 redis-cli GET icici:bal:9003
"341244.80"
```

Step 6 · Read the primary's log - this is the full sync happening

```
$ docker logs redis-primary | tail -6
Replica 172.18.0.3:6379 asks for synchronization
Starting BGSAVE for SYNC with target: replicas sockets
Diskless rdb transfer, done reading from pipe, 1 replicas still up.
Synchronization with replica 172.18.0.3:6379 succeeded
```

Solution · What just happened, mechanically

The replica connected, sent PSYNC with a replication ID it did not yet have, and the primary - having no backlog history to offer - forked itself exactly the way BGSAVE does and streamed the RDB straight over the replication socket rather than writing dump.rdb to disk first (*diskless*). The replica loaded it, and every write since has been streamed to it as it happens.

CHECKPOINT You have a working primary/replica pair, you have watched the exact log lines a full sync produces, and you have proved data written on one appears on the other. Keep both containers running - the rest of the labs build on this pair.

LAB R.2 · 25 MINUTES

Read-Only Replicas, and WAIT for Durability

Objective	Prerequisite
Prove a replica refuses writes by design, then use WAIT to confirm a write actually reached it before you move on.	Lab R.1 complete - redis-primary and redis-replica1 still running.

Step 1 · Try to write directly to the replica

```
$ docker exec redis-replica1 redis-cli SET icici:hack "should fail"
(error) READONLY You can't write against a read only replica.
```

WATCH OUT replica-read-only yes is the default. Without it, a value written straight to a replica would simply be overwritten the next time the primary sends a write for that key - a silent, confusing bug. The refusal is the safer behaviour.

Step 2 · Confirm reads still work

```
$ docker exec redis-replica1 redis-cli GET icici:bal:9003
"341244.80"
```

Step 3 · A second replica, so WAIT has something to count

```
$ docker run -d --name redis-replica2 \
  --network icici-repl-net -p 7003:6379 \
  redis:7-alpine redis-server --replicaof redis-primary 6379
$ docker exec redis-primary redis-cli INFO replication | grep connected_slaves
connected_slaves:2
```

Step 4 · WAIT for both replicas to acknowledge a write

```
$ docker exec redis-primary redis-cli SET icici:wait:test "durable write"
OK
$ docker exec redis-primary redis-cli WAIT 2 1000
(integer) 2
```

WAIT 2 1000 means: block for up to 1000 ms until 2 replicas have acknowledged the current write offset, then return how many actually did. Here both already had.

Step 5 · See WAIT time out honestly

```
# ask for more replicas than exist - it cannot lie
$ docker exec redis-primary redis-cli WAIT 5 500
(integer) 2          # only 2 replicas exist, so 2 is the ceiling
```

Solution · What WAIT does and does not do

WAIT does not make replication synchronous - the SET in Step 4 had already returned OK to the client before WAIT was even called. What WAIT gives you is a way to *check*, after the fact, how durable a recent write is before you tell a caller it succeeded - useful for anything where losing that one write on a primary failure would matter.

CHECKPOINT You can explain why a replica refuses writes, and you have used WAIT to turn an assumption about durability into a measured fact.

LAB R.3 · 20 MINUTES

Partial Sync After a Network Blip

Objective	Prerequisite
Cut a replica off from the network briefly, bring it back, and read the primary's log to see it resume without a full RDB transfer.	Lab R.2 complete - three containers running.

Step 1 · Check the sync counters before you start

```
$ docker exec redis-primary redis-cli INFO stats | grep -E "sync_full|sync_partial"
sync_full:2
sync_partial_ok:0
sync_partial_err:0
```

Step 2 · Disconnect one replica from the network

```
$ docker network disconnect icici-repl-net redis-replica2
```

Step 3 · Write on the primary while it is cut off

```
$ docker exec redis-primary redis-cli SET icici:during:blip "missed by replica2"
OK
```

Step 4 · Reconnect it

```
$ docker network connect icici-repl-net redis-replica2
```

Step 5 · Read the primary's log for the resync

```
$ docker logs redis-primary --tail 5
Replica 172.18.0.4:6379 asks for synchronization
Partial resynchronization request from 172.18.0.4:6379 accepted.
Sending 0 bytes of backlog starting from offset 266.
```

Step 6 · Confirm the data arrived, and the counters moved

```
$ docker exec redis-replica2 redis-cli GET icici:during:blip
"missed by replica2"
$ docker exec redis-primary redis-cli INFO stats | grep -E "sync_full|sync_partial"
sync_full:2          # unchanged - no full transfer happened
sync_partial_ok:1    # this is the resync you just watched
```

Solution · Why this matters operationally

A container restart, a brief network partition, a rolling deploy that bounces a pod for two seconds - none of these need to cost a full RDB transfer if the gap is shorter than the primary's `repl-backlog-size` can cover. `sync_full` climbing on a healthy system usually means either the backlog is too small for how long your restarts take, or something is repeatedly forcing replicas to lose their replication ID (see Lab R.6).

CHECKPOINT You have distinguished a partial resync from a full one by reading the log and the counters, not by guessing.

LAB R.4 · 30 MINUTES

docker-compose: a 3-Node Topology by Name

Objective	Prerequisite
Run the same three-node shape from a single compose file, and prove it survives the primary being force-recreated.	Lab R.3 complete. Uses replication_docker/docker-compose.yml from this pack.

Step 1 · Clean up the manually-wired containers

```
$ docker rm -f redis-primary redis-replica1 redis-replica2
$ docker network rm icici-repl-net
```

Step 2 · Bring the stack up

```
$ cd replication_docker
$ docker compose up -d
$ docker compose ps
NAME                STATUS
icici-redis-primary Up (healthy)
icici-redis-replica1 Up
icici-redis-replica2 Up
```

depends_on: condition: service_healthy on both replicas means compose does not even start them until the primary's own healthcheck passes - there is no race to connect to a primary that is not accepting connections yet.

Step 3 · Confirm both replicas attached

```
$ docker exec icici-redis-primary redis-cli INFO replication | grep connected_slaves
connected_slaves:2
```

Step 4 · Write, then force-recreate the primary mid-session

```
$ docker exec icici-redis-primary redis-cli SET icici:compose:repl "via compose"
OK

$ docker compose up -d --force-recreate redis-primary
Container icici-redis-primary Recreated
Container icici-redis-primary Started
```

This container now has a **different IP address** on the compose network. Nothing was run against either replica.

Step 5 · Prove the replicas reattached on their own

```
$ docker exec icici-redis-replica1 redis-cli INFO replication | grep -E "role|master_link"
role:slave
master_link_status:up           # measured - reattached with no manual step
$ docker exec icici-redis-replica1 redis-cli GET icici:compose:repl
"via compose"                  # the data was still there
```

Solution · Why the recreate did not break anything

The replicas were told `--replicaof redis-primary 6379` - a hostname, not an address. Docker's embedded DNS inside the compose network resolves that hostname to whichever container currently owns it, so the moment the new primary container answered on that name, the replicas' existing retry loop found it and resynced - no config edit, no restart of the replicas, no human intervention.

CHECKPOINT You have a self-healing three-node topology defined in one file, and you have watched it survive its primary being destroyed and rebuilt.

LAB R.5 · 30 MINUTES

Manual Failover: Promote a Replica

Objective	Prerequisite
Simulate a dead primary, promote a replica by hand, and repoint the surviving sibling at the new one.	Lab R.4 complete - the compose topology running.

Step 1 · Kill the primary

```
$ docker stop icici-redis-primary
```

Step 2 · Watch a replica notice

```
$ docker exec icici-redis-replica1 redis-cli INFO replication | grep -E "role|master_link"
role:slave
master_link_status:down
```

WATCH OUT Nothing else happens by itself. No alert fires, nothing gets promoted - that is the entire subject of the next slide in the deck, and it is why this lab exists: to make the gap concrete before you ever rely on Sentinel or a managed service to close it.

Step 3 · Promote replica1

```
$ docker exec icici-redis-replica1 redis-cli REPLICAOF NO ONE
OK
$ docker exec icici-redis-replica1 redis-cli INFO replication | grep -E "role|connected"
role:master
connected_slaves:0
```

Step 4 · Repoint the surviving sibling

```
$ docker exec icici-redis-replica2 redis-cli REPLICAOF icici-redis-replica1 6379
OK
$ docker exec icici-redis-replica2 redis-cli INFO replication | grep -E
"role|master_host|master_link"
role:slave
master_host:icici-redis-replica1
master_link_status:up
```

Step 5 · Prove the new primary actually works

```
$ docker exec icici-redis-replica1 redis-cli SET icici:after:failover "new primary accepted
this"
OK
$ docker exec icici-redis-replica2 redis-cli GET icici:after:failover
"new primary accepted this"          # propagated to the sibling
```

Solution · The four decisions this lab made for you, by hand

Decision	Who made it here	Who would make it with Sentinel
Detect the primary is down	You, reading INFO replication	Sentinel, via periodic PING

Decision	Who made it here	Who would make it with Sentinel
Which replica to promote	You, picked replica1 arbitrarily	Sentinel, by priority and offset
Run REPLICAOF NO ONE	You, typed it	Sentinel, automatically
Tell the application	Nobody - it still has the old address	Sentinel, via its client protocol

CHECKPOINT You have performed a real failover and can explain, precisely, which of the four jobs above a human just did that an automated system would normally do instead.

LAB R.6 · 20 MINUTES

The Old Primary Rejoins - and Why It Costs a Full Resync

Objective	Prerequisite
Bring the failed node back into the topology cleanly, and read the exact log line that explains why it cannot simply pick up where it left off.	Lab R.5 complete.

Step 1 · Restart the old primary and point it at the new one

```
$ docker start icici-redis-primary
$ docker exec icici-redis-primary redis-cli REPLICAOF icici-redis-replica1 6379
OK
```

Step 2 · Watch it try - and fail - a partial resync first

```
$ docker logs icici-redis-primary --tail 8
Trying a partial resynchronization (request 8506b36a...:266).
Partial resynchronization not accepted: Replication ID mismatch
(Replica asked for '8506b36a...', my replication IDs are
'91e667d0...' and 'a889f6f5...')
Full resync from master: 91e667d00448d1597864c36e8d480603c61fe101:388
```

Step 3 · Confirm it is healthy again

```
$ docker exec icici-redis-primary redis-cli INFO replication | grep -E "role|master_link"
role:slave
master_link_status:up
$ docker exec icici-redis-primary redis-cli DBSIZE
(integer) 6
```

Solution · Read the replication ID mismatch, field by field

Every primary has a `master_replid`. When `icici-redis-primary` ran (implicitly, by having been a primary before this whole lab) it carried the original ID. The moment `icici-redis-replica1` was promoted in Lab R.5, it generated a brand-new replication ID for itself, and kept the old one only as a secondary ID valid up to the offset where the promotion happened. When `icici-redis-primary` came back asking to resume from an offset under its own old ID, that ID was no longer the current one on the node it was asking - a partial resync from there would mean guessing at data the new primary never confirmed. Redis refuses to guess and does a full resync instead.

TIP This is also why every earlier lab that measured `sync_full` counted 2, not 1 or 0 - the first `replica1` and `replica2` attachments were full syncs by necessity (nothing to resume from), and this rejoin was a second, unavoidable one because a promotion had happened in between.

Cleaning up

```
$ docker compose down -v
$ docker volume ls | grep icici_redis          # confirm nothing is left
```

CHECKPOINT You can read a real replication-ID-mismatch log line and explain, from first principles, why it is correct behaviour rather than a bug.

What you should be able to do now

- Turn any Redis container into a replica with one flag, and read a primary's log to identify a full sync as it happens.
- Explain why a replica refuses writes by default, and use WAIT to confirm a write's durability instead of assuming it.
- Distinguish a partial resync from a full one by reading INFO stats and the primary's log, and explain what determines which one happens.
- Run a self-healing multi-node topology under docker-compose, wired entirely by container name, and prove it survives its primary being destroyed and rebuilt.
- Perform a manual failover end to end, and name the four jobs a human just did that an automated system like Sentinel would normally do instead.
- Read a real 'Replication ID mismatch' log line and explain why Redis chooses a full resync rather than guess.

NEXT Next: open 07_Redis_Replication.pptx and re-run any command on this page that surprised you - everything in that deck was captured from the same kind of session as this guide.